Clase 2
Metodologías de Desarrollo de Software

Objetivos de la Clase

- Comprender *Contexto y Metodologías* de desarrollo
- Analizar *Scrum en Detalle* como ejemplo de metodología ágil
- Explorar *Herramientas y Colaboración* en un proceso de desarrollo

Agenda

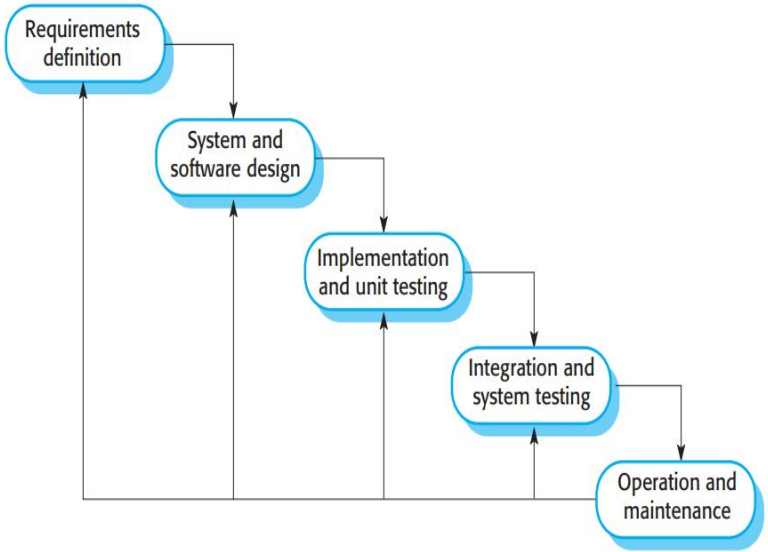
1. **Contexto y Metodologías**
2. Scrum en Detalle
3. Herramientas y Colaboración
4. Buenas Prácticas

Importancia de los procesos

- Los procesos de desarrollo estructuran el trabajo en proyectos de software
- Permiten coordinar equipos y roles
- Ayudan a reducir riesgos y errores
- Determinan velocidad y calidad de entrega

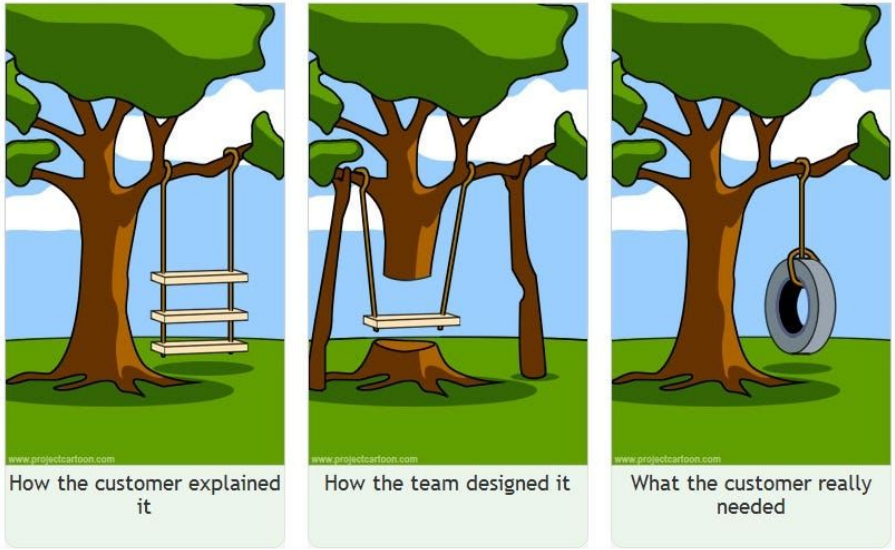
Waterfall: visión general

- Modelo lineal, secuencial de desarrollo
- Útil en proyectos predecibles y con pocos cambios
- Se inspira en procesos de ingeniería tradicionales



Waterfall: Limitaciones

- **Riesgo de desalineamiento**: dado que el cliente solo ve el sistema en su estado final introducir cambios se torna costoso.
- **Poca adaptación** en entornos dinámicos donde los requerimientos o el contexto cambian.



Introducción a Agile

Surge como alternativa a waterfall.

El Agile Manifesto (2001) introdujo 12 ideas fundamentales y 4 valores clave:

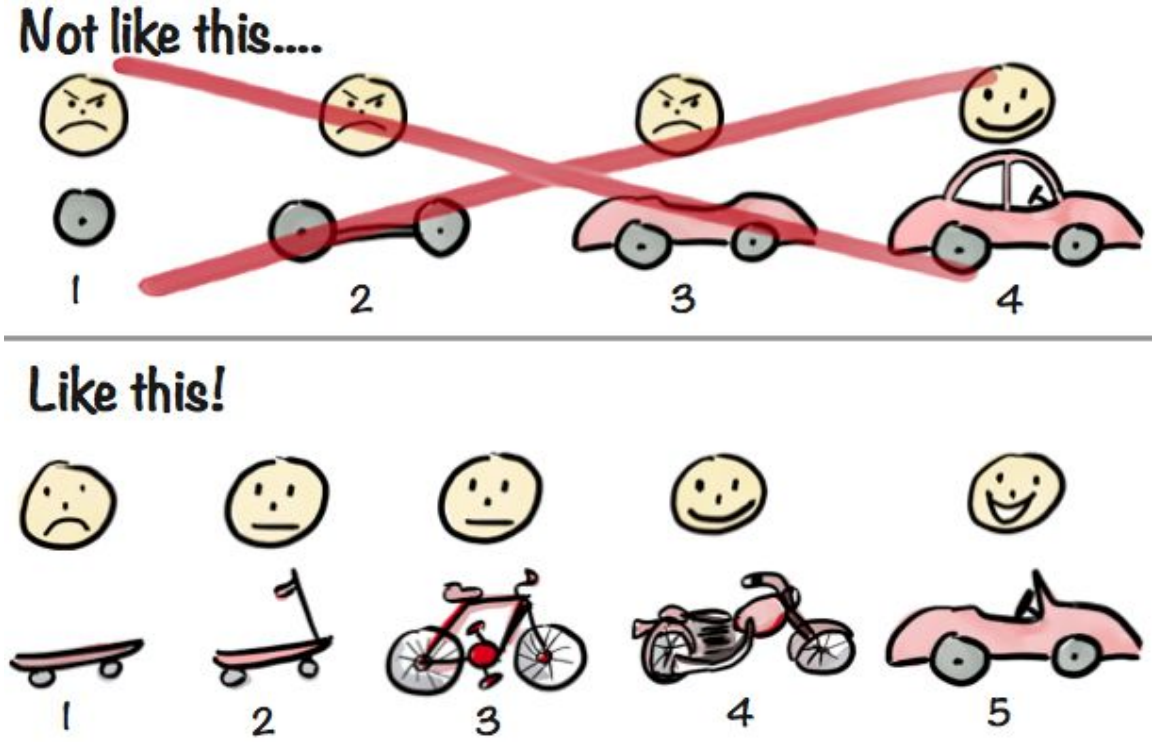
- **Personas** > procesos
- Software **funcional** > documentación
- **Colaboración** > contratos
- Responder al **cambio** > seguir un plan



Características de Agile

- Desarrollo **iterativo** e **incremental**
- **Entregas pequeñas y frecuentes** enfocadas al valor al cliente
- Comunicación directa y constante con **usuarios/clientes**
- Foco **simplicidad** y **adaptabilidad**
- Uso intenso de **integración continua**
- Se busca mantener un **ritmo sostenible**

Características de Agile



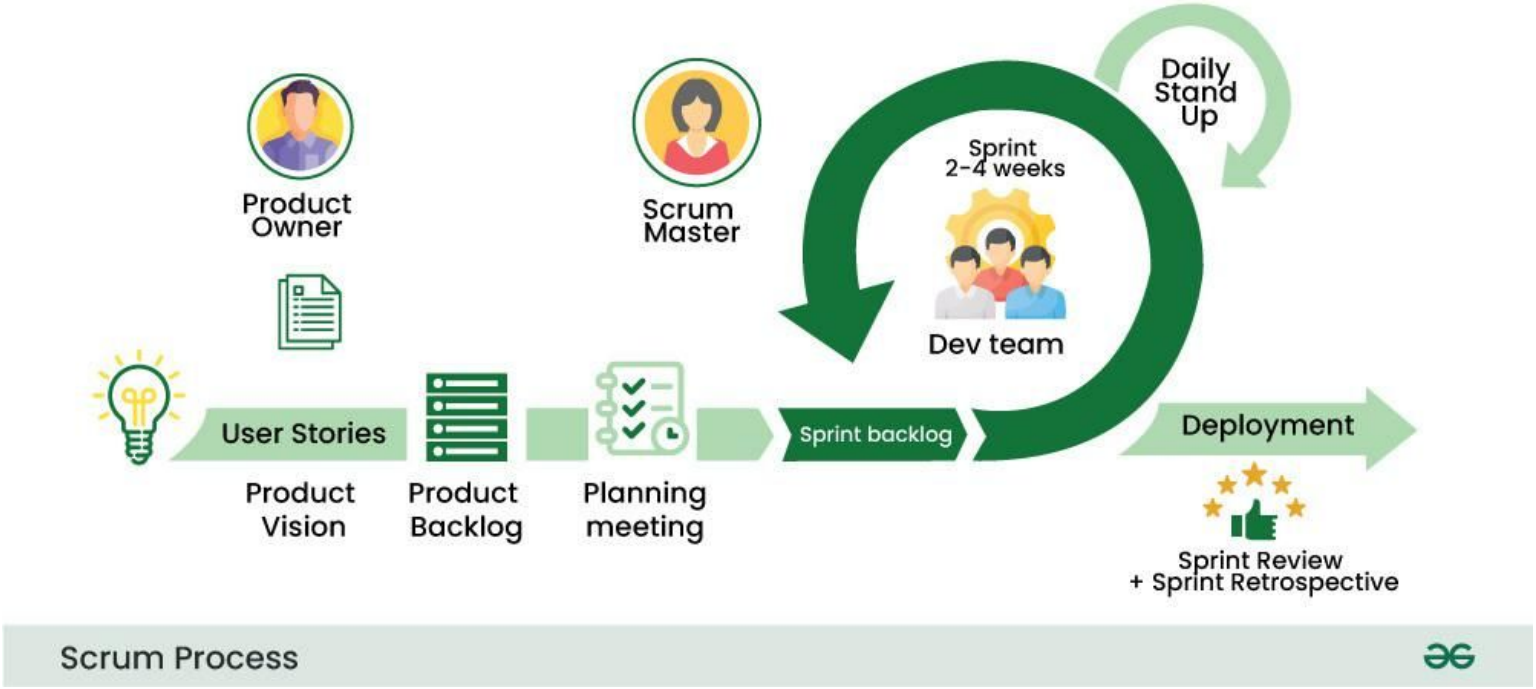
Agenda

1. Contexto y Metodologías
2. **Scrum en Detalle**
3. Herramientas y Colaboración
4. Buenas Prácticas

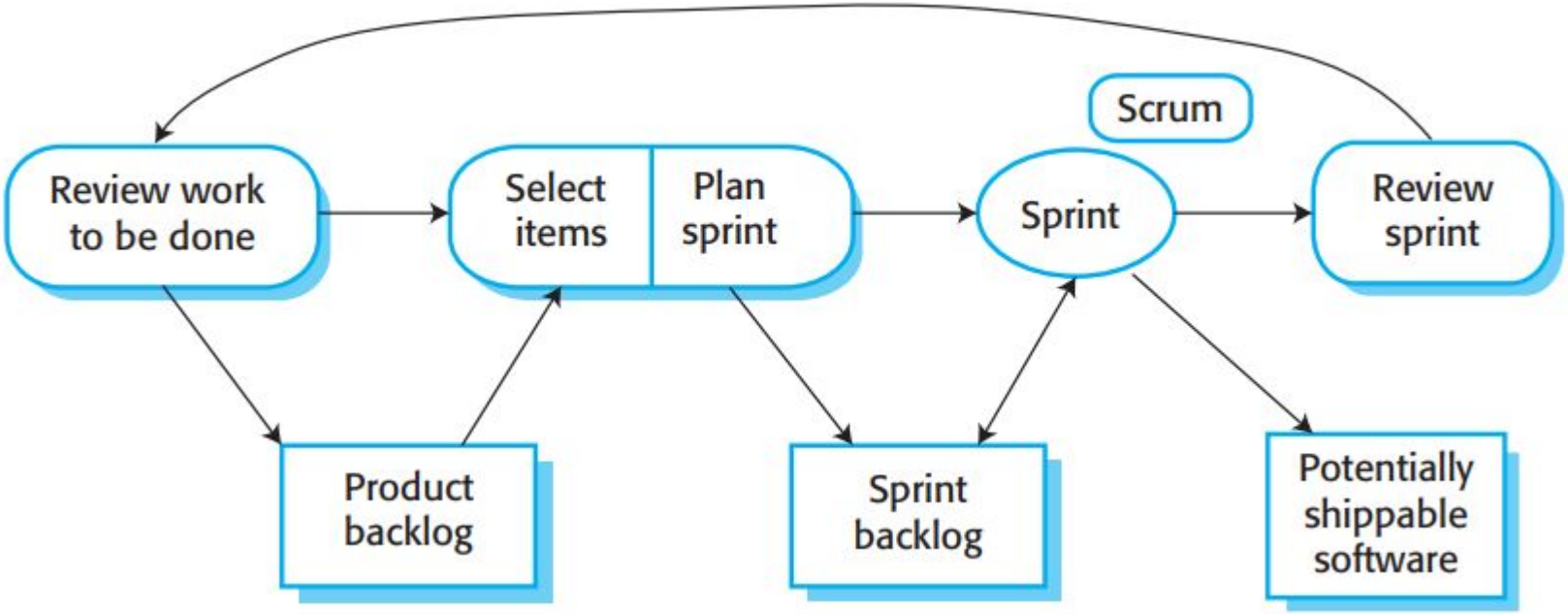
Scrum: visión general

- **Framework** para gestión de proyectos
- Organizado en **sprints cortos** (2–4 semanas)
- Fomenta transparencia, inspección y adaptación
- Basado en **roles**, **artefactos** y **eventos**

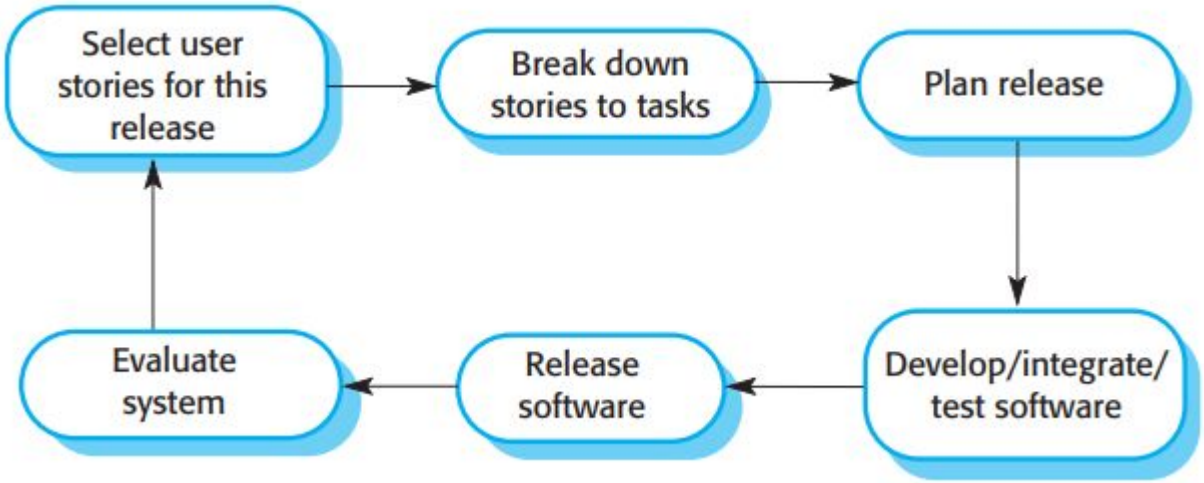
Scrum: visión general



Scrum: Sprints Cycle



Scrum: Sprints Cycle



Roles: Product Owner

- **Responsable de maximizar valor del producto**
- **Representa la visión de negocio** de clientes y stakeholders
- Toma decisiones sobre **alcance y prioridades** del Product Backlog

Roles: Scrum Master

- Garantiza la correcta **aplicación de Scrum**
- **Elimina impedimentos** que bloquean al equipo
- **Protege al equipo** de distracciones externas
- **Fomenta la mejora continua** (retrospectives)

Roles: Dev Team

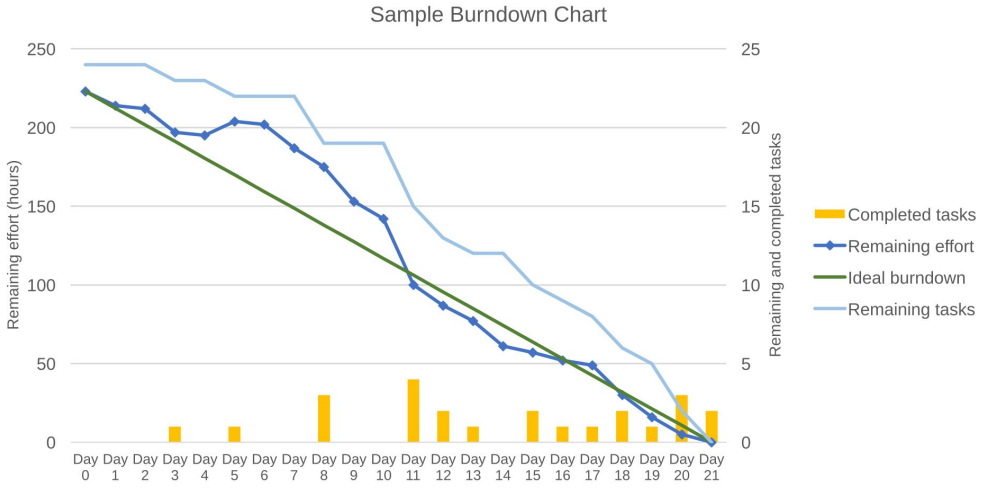
- **Autoorganizado** y **multidisciplinario** (frontend, backend, ML, etc)
- En cada sprint construyen **software incrementalmente**
- Comparten **responsabilidad** del resultado
- **Colabora en estimación y planificación** con el Scrum Master y Product owner.
- **Mantienen calidad técnica y buenas prácticas**

El Product Backlog

- Lista priorizada de funcionalidades y tareas
- Controlado por el **Product Owner**
- Evoluciona con el tiempo
- Fuente principal de planificación

El Sprint Backlog

- **Subconjunto del Product Backlog** para el sprint actual
- Incluye tareas estimadas y asignadas
- Permite medir avance del equipo
- Base para el **Daily Scrum**



Ciclo de vida de un Sprint

- Inicio: planificación con el Product Backlog
- Selección de historias al Sprint Backlog
- Desarrollo colaborativo en iteraciones cortas
- Cierre: revisión + retrospectiva

Nota a considerar

- En la práctica conviene tomar Agile como una **caja de herramientas**
- La idiosincrasia de las organizaciones y equipos lleva a que sea necesario **experimentar** hasta llegar a la metodología que mejor se adecúe a cada uno.

Caso de Implementación de Agile

- Ceremonias
  - Kick off con clientes
  - Bi-weeklies con clientes
  - Internal syncs con
  - Daily standups async
  - Retrospectivas
- Comunicacion
  - Project proposal
  - Live-documents de arquitectura
  - Architecture Decision Records (ADRs)
  - Equipos de trabajo integrados vía Slack/Teams
  - Identificación clara de enablers: data stewards, evaluadores de negocio, owners de KPIs para mejorar, etc.
  - Task tracking compartido con el cliente

Agenda

1. Contexto y Metodologías
2. Scrum en Detalle
3. **Herramientas y Colaboración**
4. Buenas Prácticas

Task tracking: propósito

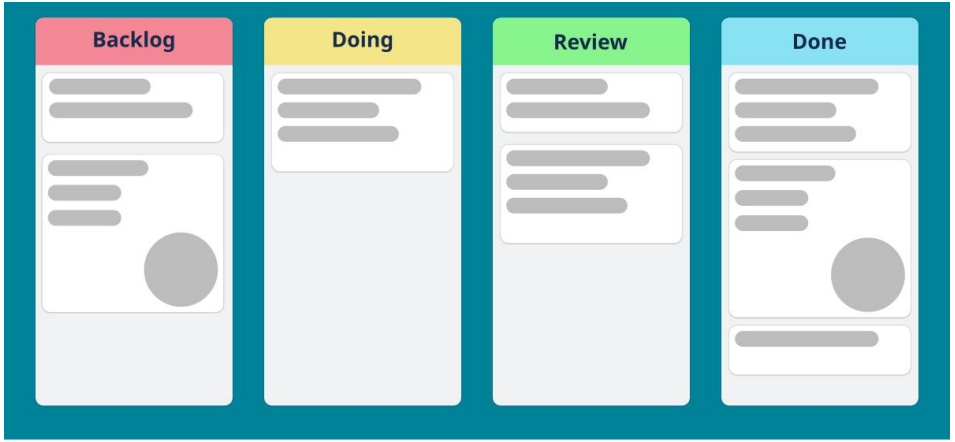
- Organizar y visualizar tareas pendientes
- Mejorar transparencia y responsabilidad
- Identificar bloqueos y dependencias
- Facilitar comunicación entre roles

Ejemplos de herramientas

- Jira: gestión ágil, integración con Git
- Trello: tableros Kanban simples
- ClickUp: tareas + docs + chat
- GitHub Projects: integrado a repositorios

Tableros Kanban y Scrum boards

- Es una forma de visualizar el estado de las tareas
- Columnas típicas: Backlog, Doing, Review/Test, Deployment, Done
- Se suele definir un WIP limit para evitar tener demasiados frentes abiertos.



Agenda

1. Contexto y Metodologías
2. Scrum en Detalle
3. Herramientas y Colaboración
4. **Buenas Prácticas**

Retrospectivas

- ¿Qué es?
  - Es una reunión del equipo para reflexionar sobre el trabajo realizado y definir mejoras continuas.
- ¿Cuándo se hace?
  - Al finalizar un sprint o una etapa clave de un proyecto.
- ¿Quienes participan?
  - Todo el equipo de desarrollo

Preguntas que responde cada participante por separado:

- ¿Qué salió bien / Qué me gustó?
- ¿Qué no salió tan bien?
- ¿Aprendizajes / Cambios a realizar?
- ¿Qué me genera dudas?
- ¿Por qué estoy agradecido?

Blameless Post Mortem

- Blameless: **Se parte de la buena fe**, evitar buscar responsables individuales centrándose en hechos. Busca fomentar una cultura abierta y de aprendizaje continuo.

- Post mortem: Es una **revisión estructurada de un incidente**.
  Documenta:
  - **Summary** => ¿Qué pasó? ¿Cuál fue el impacto? ¿Cuánto duró?
  - **Leadup** => ¿Qué llevó a que se de el incidente?
  - **Response** => ¿Quién responde? ¿Cuándo? ¿Cómo? ¿Hubo blockers?
  - **Root Cause** => ¿Cuál causa raíz evitaría recurrencia del incidente?
  - **Recurrence** => ¿Ocurrió esto anteriormente?
  - **Corrective actions** => ¿Qué acciones se tomarán para evitar o mitigar el problema?

Blameless Post Mortem (ejemplo)

Proyecto de ASR fallido:

- **Summary**
  - Solapamiento train-test; métricas inválidas; impacto 1.5 meses
- **Leadup**
  - Dataset mal dividido; supuestos no validados; confianza excesiva del stakeholder al no compartir las métricas que se utilizaban.
- **Response**
  - De inmediato se reportó el problema y se volvieron a correr los entrenamientos con datasets limpios.
- **Root Cause**
  - Supuestos no validados sobre solapamiento en splits
- **Recurrence**
  - No reportado; posible por ser silent failure
- **Corrective actions**
  - Revisiones pipeline, partición con hash, formalizar requerimientos de métricas.

Dudas?